

ServiceOps complete platform manual

Version 1.27.22 · Visual walkthrough edition

Contents

1. Purpose and operating model	4
2. Roles and teams	4
3. End-user guide	4
REST API documentation	4
Governed record projections	4
Browser request security	5
Database migration gate	5
Declarative action and field authorization	6
REST API v1	6
Responsive PWA	6
Durable integrations and monitoring	7
Sign in	7
Workflow state integrity	7
ITIL related-record model	7
Ticket history and approval-safe change revisions	8
Disposable full-function test fixture	8
Navigation	9
Incidents	9
Requests and catalog	10
Changes	10

Knowledge, CMDB, assets, and boards	10
---	----

3A. Visual walkthrough 10

Dashboard	11
Incidents list	11
Incident detail	12
Changes list	13
Requests & catalog	14
Requested Item (RITM) detail	16
Open work	16
My tasks	17
Visual task board	18
CMDB & service map	19
Org chart	21
Manager portal	22
Analytics	23
My approvals	25
Notifications	25
Knowledge base	26
Administration → System settings	27
Users & roles	29
Audit log	29
ITIL configuration	31

3B. Common task walkthroughs 33

Report and resolve an incident	33
Submit a Normal change through approval	33
Order an item from the service catalog	33
Review team performance as a manager	34

4. Deployment decision	34
5. Docker installation	34
6. Kubernetes prerequisites	34
7. Kubernetes installation	34
8. Kubernetes chart controls	35
9. Identity configuration	35
Local administrator	35
AD/LDAP	35
Keycloak	35
10. Post-deployment system settings	36
Priority and SLA policy	36
Declarative workflow operations	36
Bootstrap credential retirement	37
Release evidence	37
11. Security operations	38
12. Backup and recovery	38
13. Upgrades and rollback	38
14. Monitoring and SLOs	39
15. Incident response	39
16. Production acceptance checklist	39

See ``docs/MASTER_REFERENCE.md`` for a single merged document that also includes the governed backlog, traceability matrix, ITIL hierarchy reference, BP-001 blueprint traceability, REST API reference, and a fresh whole-application audit alongside this manual — useful as one-stop context for an AI agent or new engineer.

1. Purpose and operating model

ServiceOps is an enterprise service-management platform for requesters, fulfillers, team managers, CCB members, and platform administrators. It includes incident, request, problem, change, catalog, knowledge, CMDB, asset, SLA, approval, audit, analytics, and enterprise workspaces.

No deployment mechanism can eliminate every infrastructure or operator failure. The ServiceOps production standard therefore uses prevention, validation, observability, tested recovery, least privilege, immutable releases, and documented rollback rather than claiming infallibility.

2. Roles and teams

Role	Normal responsibilities
Requester	Submit and track requests and incidents, search knowledge
Agent	Triage, assign, fulfill, document, and resolve operational work
Manager	Agent work plus team oversight and manager approvals
CCB member	Review planned changes, risk, evidence, schedule, and backout plan
Administrator	Identity, ITIL configuration, CMDB, audit, and platform operation

CoreApps, Database, Network, Windows, Unix, and SSD are standard fulfillment teams. Administrators can map AD groups such as ``gg_unix`` to the Unix team; membership reconciles whenever the user logs in through AD/LDAP. Team-manager appointment and CCB approval authority remain explicit administrative decisions. Only active users granted ``CCB approver`` authority participate in new CCB approval gates. Normal and emergency changes follow manager assessment and CCB authorization.

Operational ownership is separate from approval authority. Every incident, service request, and change has one owning IT fulfillment team. An active member of that team, its configured manager, or a platform administrator can change operational fields. Assignees must remain active members of the owning team. Active members and managers of every IT fulfillment team can read all incidents and changes through lists, search, direct URLs, dashboards, analytics, task boards, history and attachments. This shared visibility supports triage, handoffs and operational awareness, but only the owning team or a platform administrator can change the parent ticket. A named approver can see the record needed for the decision, but CCB membership grants approval authority only and does not grant cross-team operational control. Administrator actions remain audited.

3. End-user guide

REST API documentation

The complete integration guide is maintained in ``docs/API_REFERENCE.md`` and is served by the running application at ``/api/v1/docs``. The machine-readable OpenAPI 3.1 contract is ``/api/v1/openapi.json``. It documents authentication, scopes, tenant and team authorization, field projections, idempotency, pagination, ticket operations, workflow triggers, monitoring ingestion, errors, retries, cURL and Python examples, and the explicit current compatibility boundary.

Governed record projections

Record visibility and field visibility are separate controls. Central tenant-aware object policies decide whether a user can discover or access a record. The validated Git-backed policy in `config/field_projections.json` then decides which fields may leave the application for that resource and audience.

The registry covers every supported record REST response, signed audit export, JSON search result, monitoring acknowledgement, workflow acknowledgement and interactive mutation acknowledgement. Unknown resources, unknown audiences, duplicate fields and fields outside a resource allowlist fail application startup. Requester ticket projections exclude internal assignment details; internal projections are limited to agent, manager and administrator audiences. Adding a new record API or export requires a reviewed registry binding and adversarial projection test.

Browser request security

Every state-changing browser request requires a session-bound CSRF token. ServiceOps inserts tokens into all rendered POST forms and supplies the token to interactive workspace actions through a dedicated request header. Tokens rotate after authentication. Missing, expired or mismatched tokens fail closed with HTTP 400 and make no data change.

Session cookies are `HttpOnly` and `SameSite=Lax`, and `SESSION_COOKIE_SECURE` defaults to `true` everywhere (Docker Compose, Helm, and the RPM package), not only in a production Helm profile. Outside test mode, the application refuses to start with insecure cookies unless `ALLOW_INSECURE_SESSION_COOKIES=true` is also set explicitly; that escape hatch exists only for an isolated plain-HTTP localhost/development deployment and must never be set in production. `SESSION_LIFETIME_MINUTES` controls the bounded session lifetime.

A request whose authenticated user has no resolvable `tenant_id` is logged out and rejected with HTTP 403 rather than silently defaulted to a tenant -- tenant resolution fails closed.

A Content-Security-Policy header (`default-src 'self'`, restricting script, style, image, font, and connection sources to the same origin) is sent on every response.

Database migration gate

Production schema changes are controlled by Alembic revisions. Compose runs the migration gate before initialization. Kubernetes uses a dedicated Helm pre-install/pre-upgrade migration Job; normal application replicas set `AUTO_MIGRATE=false` and refuse to start unless the database is already at the required revision.

Revision `20260726_0001` adopts a complete existing ServiceOps schema without rewriting operational records, or creates the schema for a fresh installation. Its downgrade is intentionally non-destructive: removal of an adopted baseline requires restoring the validated pre-migration backup. Every later schema revision must provide and test its own one-revision downgrade.

Revision `20260726_0002` creates the installation's default tenant and assigns all existing tenant-owned root records to it without deleting or recreating operational data. The application constrains ticket, request, enterprise, knowledge, asset, catalog, CMDB, notification, administration, approval, and global-search roots to the signed-in user's tenant. Cross-tenant direct identifiers return HTTP 404. Automated migration evidence covers fresh installation, existing-schema adoption, one-revision downgrade, roll-forward, row preservation, and tenant backfill. A deployment still requires a validated backup and an environment-specific rehearsal before production promotion.

For bundled PostgreSQL, run the guarded rehearsal before promoting an upgrade:

```
./serviceops rehearse-migrations 100000
```

The command clones the installed database into a database whose name must end in `_migration_rehearsal`, adds the requested number of isolated representative records, performs a one-revision downgrade and roll-forward, and compares table counts plus stable identity/content fingerprints. A cleanup trap removes the clone on success or failure. The command refuses external-database mode because those rehearsals must use an isolated database supplied by that provider.

Revision `20260726_0003` extends every audit event with a UUID, request correlation identifier, source context, integrity algorithm, previous-event hash, and event hash. Existing events are deterministically sealed into a tenant-specific legacy SHA-256 chain; new events use HMAC-SHA-256 with the deployment integrity key.

PostgreSQL and SQLite triggers reject audit UPDATE and DELETE operations at the database boundary. The administrator audit page verifies the entire tenant chain before displaying its status. Audit export is blocked if verification fails and otherwise returns ordered JSON evidence with a detached signature in ``X-ServiceOps-Audit-Signature``.

Revision ``20260726_0011`` gives every HMAC event a signing-key identifier. Administrators can rotate forward from the audit page only after full-chain verification succeeds. Historical secrets remain encrypted for verification; neither events nor hashes are rewritten. The export response identifies its signing key in ``X-ServiceOps-Audit-Key-ID``. ``AUDIT_INTEGRITY_KEY_FILE`` supports an initially mounted secret and takes precedence over the environment value.

The same page governs a minimum seven-year retention period, legal hold, and the requirement for external immutable evidence. Primary audit rows are never automatically purged. When audit streaming is enabled, each new audit event is committed transactionally to the durable outbox. Only active ``siem`` connections receive ``audit.created`` events; ordinary webhooks and Teams connections cannot receive this security stream. Delivery uses HTTPS, event identifiers, timestamps and HMAC signatures, with bounded retry and delivery evidence. The organization must still validate its actual immutable SIEM/WORM destination and retention controls.

Protect ``AUDIT_INTEGRITY_KEY`` as a rotated secret independent from database operator access. When it is absent, ServiceOps uses the settings encryption key and finally the application secret. Changing the effective key without a governed chain rollover makes prior HMAC events unverifiable. Database append-only enforcement does not replace external immutable retention; export signed evidence to the organization's governed WORM or SIEM destination.

Declarative action and field authorization

``config/authorization.json`` is the Git-controlled source for the initial role/action vocabulary. It declares discover, read, public/internal comment, create, update, assign, accept, transition, resolve, close, reopen, approve, delegate, relate, export, report, delete, purge, configure, administer, and security-administration actions. Startup fails if the policy is malformed or a role contains an unknown action. Browser handlers and future REST handlers use the same ``serviceops_core.security`` interface.

Object visibility remains relationship-aware and tenant-aware. Field projection is a separate decision: requesters viewing their own incident receive its public description, public comments, public attachments, state, priority, category, assignment summary, and ownership summary. They do not receive internal ticket history, work notes, change risk/plans, approval records, SLA internals, major incident coordination, affected-CI internals, operational tasks, or checklists. Active fulfillers retain the internal projection according to their object and team authorization. Client-supplied fields never expand that projection.

REST API v1

Administrators create and revoke REST identities under ****Administration → API clients****. Each client is bound to one active user in the same tenant and inherits that user's object, relationship, field, and action authorization. Explicit scopes further restrict the client. Plaintext bearer tokens are shown only in the creation response; ServiceOps stores an HMAC hash and prefix, never the recoverable token. Protect the independently generated ``API_TOKEN_PEPPER``; changing it revokes every existing token by design.

The initial ``/api/v1`` contract provides access-aware cursor-paginated ticket listing, individual ticket retrieval, incident creation, controlled ticket updates, and an OpenAPI 3.1 discovery document. Unsafe operations require an ``Idempotency-Key``. Repeating identical content replays the stored JSON result; reusing a key with different content fails with HTTP 409. API errors include the correlated request identifier. Writes enter the append-only audit chain with the acting user and API client identifier.

Responsive PWA

ServiceOps publishes a dynamic company/instance manifest and root-scoped service worker. The worker caches only CSS and JavaScript shell assets. It does not intercept or cache HTML, authentication, API responses, tickets, requests, attachments, or other operational records. Installation outside localhost requires HTTPS. Offline operational records remain explicitly deferred until encrypted device storage, remote revocation, and data-loss policy are

approved.

Durable integrations and monitoring

Revision `20260726\_0005` adds a PostgreSQL-backed outbox, per-channel delivery evidence, encrypted webhook/Teams connections, authenticated monitoring sources, and deduplicated monitoring events. Application transactions create notifications and outbox events together. The separate Compose/Kubernetes worker claims due events with `FOR UPDATE SKIP LOCKED`, delivers them, records each attempt, retries with bounded exponential delay, and moves an event to `Dead` after five failed processing attempts. A successful channel is not sent again during retries of another channel.

SMTP is configured post-installation under platform settings. STARTTLS is enabled by default; hostname, port, account, encrypted password, and from address remain administrator-controlled. Signed webhooks use HTTPS and carry event ID, timestamp, and `HMAC-SHA-256` signature headers. Teams connections use the same durable worker with Teams-compatible message payloads. Literal loopback, private, link-local, and non-HTTPS webhook targets are rejected at configuration time. At delivery time the application re-resolves the destination hostname and rejects it if it now resolves to a non-global address, disables automatic redirect-following, and manually re-validates and re-resolves the target on every redirect hop (up to three), closing most of the DNS-rebinding/redirect-SSRF gap without relying solely on network egress controls. A narrow TOCTOU window remains between that re-resolution and the HTTP client's own connection-time DNS lookup; true IP pinning or a controlled outbound proxy is tracked as future work. Defense-in-depth egress firewall/DNS controls at the network layer are still recommended in production.

Administrators create monitoring sources under **Integrations** and bind each source to an active IT fulfillment team. The source token is displayed once and stored only as a hash. `POST /api/v1/monitoring/{source\_id}/events` requires that bearer token, validates severity and payload limits, deduplicates by source/external ID, creates an EVT record, assigns an EVTASK investigation to the configured team, maps critical/high/medium/low severity to P1/P2/P3/P4, and records ingestion in the append-only audit chain.

Sign in

The sign-in page offers the identity methods enabled by the administrator:

Workflow state integrity

Approval-derived states are controlled by the approval engine and cannot be selected manually. A change remains `Awaiting Approval` until its active manager and CCB gates complete; only then can it move from `Approved` to `In Progress`. The ticket form and visual task board use the same server-side transition policy. Administrators cannot cast another named approver's vote. After approval, the owning team or a platform administrator can start, resolve, reopen, or close the work. A member of SSD can read a Unix-owned ticket for operational awareness but cannot operate it merely because that person is a manager or CCB approver.

Enterprise records with requested approvals remain locked in `Awaiting Approval`. Catalog fulfillment tasks cannot begin until their RITM approval chain completes, and terminal records cannot be reopened through a generic update endpoint. Invalid transitions return HTTP 409 and make no data change.

ITIL related-record model

ServiceOps treats operational work as a governed network rather than a ticket chain:

Record	Purpose and supported relationships
INC	Restore service; parent/child INC, primary and affected CIs, PRB, fix CHG, caused-by CHG, converted REQ
PRB	Root cause, workaround, known error and permanent fix; many INCs, PTASKs, multiple CHGs and knowledge
PTASK	Independently assigned investigation or resolution work package under a PRB

Record	Purpose and supported relationships
CHG	Risk, authorization, schedule and overall implementation control; related INCs, PRBs, RITMs, CIs and services
CTASK	Planning, implementation, testing or review work assigned to a specific team under a CHG
REQ	User order container containing one or more RITMs
RITM	Catalog item, variables, approvals and fulfillment stage; may link to a controlled CHG
SCTASK	Independently assigned fulfillment work under a RITM, in parallel or after a predecessor

CTASK, PTASK and SCTASK are distinct records. A required open CTASK prevents its parent CHG from completing. A required open PTASK prevents its PRB from completing. A RITM completes only after all SCTASK work reaches a terminal result, and a REQ completes only after all its RITMs complete.

Major-incident coordination remains an extension of an INC. It records proposed or accepted major status, business impact, coordinator and communications; it does not create a separate incident prefix. If the administrator enables parent incident state synchronization, supported state changes propagate to linked child INCs and are recorded in each child history.

Ticket history and approval-safe change revisions

Every operational record page contains its own chronological history. The history records the actor, precise time, event, field, previous value, new value and supporting details for record creation, state/priority/assignment changes, comments, attachments, checklist actions, related records, CI links, PTASK/CTASK/SCTASK work, and approval decisions.

For a CHG, the following are material approval inputs: purpose and description, change type, risk, impact, implementation plan, test plan, backout plan, planned window and primary CI. Editing any of them:

1. preserves the previous chain and votes as historical evidence;
2. marks the previous approval chain `Superseded`;
3. increments the CHG plan revision;
4. returns the CHG to `Awaiting Approval`;
5. creates a new manager/CCB approval chain; and
6. sends an in-application reapproval notification to every configured

approver.

An old approval is therefore never silently applied to a materially different change plan.

Disposable full-function test fixture

`tools/load\_test\_fixture.py` is an explicit non-production utility and is never executed during normal startup. It must be used only after an intentional database reset. The fixture displays a permanent red warning banner and creates an administrator plus one manager/CCB approver for every IT team:

Team	Username	Password
Administration	`admin`	`admin`
CoreApps	`coreapps`	`coreapps`

Team	Username	Password
Database	`db`	`db`
Network	`network`	`network`
Windows	`windows`	`windows`
Unix	`unix`	`unix`
SSD	`ssd`	`ssd`

It also creates disposable catalog items, a configuration item, an asset, a knowledge article, a service offering, AD mappings, team-manager assignments, Service Desk memberships, and CCB approval authority. Never expose an instance containing this fixture to a network or reuse it for production data.

``tools/load_demo_dataset.py`` (added in 1.27.18) is a second, non-production loader with the same ``--confirm-non-production`` requirement and identical production-image exclusion. Instead of a minimal fixture, it builds a realistic, deep dataset for manual exploration and testing: ~19 configuration items across three business-service dependency trees (application → database → server, plus shared network and storage CIs) with ~32 CI relationships, two named users per IT team (manager + agent; managers are also added as CCB approvers), five plain requester accounts, and a representative spread of 8 incidents, 3 problems (each with problem tasks), 3 changes (Standard/Normal/Emergency, each with change tasks), 2 catalog requests, and 5 knowledge articles. Every seeded user's password equals its username. Seeded changes and problems set their ``state`` directly rather than driving the live approval engine, so they render immediately without a pending approval blocking the screen — submit a **new** change or request through the UI against this data to exercise the live approval workflow itself. Keycloak SSO, AD/LDAP, and local administrator. The local administrator is a break-glass account and should not be used for routine work.

Navigation

The left application navigator contains role-appropriate modules. The top bar contains global search, favorites, recent history, notifications, help, and preferences. The categorized Preferences workspace controls density, font scale, high contrast, reduced motion, accessible tooltips, keyboard shortcuts, date/list presentation, whether the sidebar stays pinned open, and the start page. The user name in the navigation footer opens the self-service profile, where a user can maintain their name, email, title, phone, time zone and date format. Role, active state, department, team membership, manager authority and CCB authority are not self-service fields. ServiceOps is light-only; there is no theme selector (ADR-012).

Administrators use Administration home for a capability-oriented entry point to identity, ITIL configuration, workflow, CMDB, integrations, diagnostics and analytics. Users & roles provides tenant-scoped search and an editable user record. Only administrators with ``security_administer`` may alter role, active state or department. AD-sourced team membership continues to reconcile from configured directory mappings and is not replaced by profile editing.

Incidents

Create an incident with a concise title, observable symptoms, business impact, category, and urgency. Agents classify, prioritize, assign, comment, attach evidence, execute checklists, observe SLA timers, resolve, and close it.

The incident record uses a dense, two-column operational form. Its header keeps Update and permitted Resolve actions visible, while the body exposes the number, caller, contact type, state, category/subcategory, impact/urgency, calculated priority, service offering, primary configuration item, assignment group, assignee, notification preference, short description, and description. The immutable Event history follows the form directly and records field-level old/new values with actor and timestamp. Authorization remains server-enforced: the owning team controls operational mutations even though active IT teams can read incidents.

The same task-derived interaction model is used for changes, problems and other enterprise operational records, and request containers. Every supported task record keeps its number/type identity and permitted actions in a consistent

header, presents common state/priority/requester/assignment fields in the same two-column pattern, places field-level Event history immediately below the record, and then exposes type-specific governance, tasks, approvals, SLAs, attachments, work notes and related records. List workspaces use a consistent New/search/filter bar, encoded-filter summary, record count, pagination, priority indicators and operational columns.

Requests and catalog

Select a catalog item, supply its variables and business justification, and submit. ServiceOps creates REQ, RITM, approvals, and SCTASK records. Follow the request page for approval and fulfillment status.

Each catalog item has an administrator-controlled default fulfillment route under **ITIL configuration** → **Catalog item fulfillment routing**. The generated initial SCTASK inherits that team. Laptop and Software catalog items are routed to Windows by default. Administrators can route any current or future catalog item to another active fulfillment team without changing application code. The same screen creates and edits catalog items, including name, category, description, delivery target, approval requirement, active availability and fulfillment team. New items require an explicit active fulfillment team. Legacy items without an explicit route use the active Service Desk as a controlled fallback; ordering fails closed if neither an explicit team nor Service Desk is active. Inactive items cannot be ordered through either the UI or a direct endpoint.

Catalog request visibility is need-to-know. A REQ and its RITMs are visible only to the requested-by/requested-for users, members or managers of the configured fulfillment team, teams assigned an SCTASK, specifically named approval voters, and platform administrators. The same policy protects the request list, direct URL access, dashboard counts and global search. Membership in an unrelated team, such as Unix, does not expose a Windows-routed request. Administrators retain complete visibility for platform oversight and audit.

Changes

Every change must state type, affected CI, owning team, risk, impact, planned window, implementation plan, test plan, and executable backout plan. Review conflicts before approval. CCB approval is authorization, not a substitute for technical validation or membership in the owning implementation team.

Knowledge, CMDB, assets, and boards

Search knowledge before opening duplicate work. Use the CMDB relationship view to understand service impact. Asset pages track accountable inventory. The visual task board changes the underlying ticket state and therefore remains audited and role controlled.

Release 1.27.18 enriches the Configuration Item record beyond the original name/class/environment/status/IP/owner fields: each CI now also carries a description, lifecycle state (Planned/In Use/Maintenance/Retired/Disposed, distinct from operational status), business criticality (Critical/High/ Medium/Low), serial number, vendor, model, physical/logical location, cost center, discovery source (Manual/Discovery scan/Import/API), install and warranty-expiry dates, an owning support group (in addition to the individual owner), and a free-form JSON attributes bag for anything not covered by a named column. CI-to-CI relationships are now tenant-scoped in their own right (`ci_relationship.tenant_id`) rather than relying solely on their parent/child CI's tenant, and a given relationship type between the same two CIs can no longer be created twice. Administrators manage all of this from `/cmdb` and `/cmdb/<id>/edit`; the REST CMDB auto-registration endpoint (`PUT /api/v1/cmdb/configuration-items`, `cmdb:write` scope, see `API_REFERENCE.md` §9) has **not** yet been extended to accept these new fields — it remains upsert-by-name over the original five fields only, so richer CI data must currently be entered through the web UI. Migration: `20260729_0023_cmdb_enrichment.py`.

3A. Visual walkthrough

This chapter shows every major workspace as it actually renders, captured directly from a running instance, so a new user or administrator can recognize each screen before they open it for the first time. Screenshots were captured against a local development instance seeded with realistic sample data (`tools/load_demo_dataset.py`); a production instance looks identical in layout and differs only in the records shown.

Dashboard

yo ServiceOps
YOUR COMPANY

WORKSPACE

Dashboard

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

System Administrator
Admin

Search ServiceOps

SERVICE OPERATIONS

Good day, System

Here's what's happening across your service desk.

+ Report an incident

Incidents
8
P1: 0 · P2: 2

Requests
4
View REQ / RITM →

Changes
4
View schedule →

Open work
10
Needs attention →

Assigned to me

Open tickets where you are the assignee

Nothing is currently assigned to you.

Recently updated

Your latest service activity

NUMBER	SUMMARY	STATE	PRIORITY	UPDATED
CHG0000012	emergency	Awaiting Approval	P2	Jul 29, 21:15
CHG0000011	Deploy standard laptop imaging update	Closed	P3	Jul 29, 16:14
CHG0000010	Emergency firewall rule rollback on EDGE-FW01	Implemented	P2	Jul 29, 16:14
CHG0000009	Upgrade PORTALDB01 to PostgreSQL 16.4	Scheduled	P3	Jul 29, 16:14
INC0000008	APPSRV02 high CPU utilization	Closed	P4	Jul 29, 16:14
INC0000007	SAN01 disk predictive failure alert	New	P3	Jul 29, 16:14
INC0000006	Exchange Online connector authentication errors	In Progress	P2	Jul 29, 16:14
INC0000005	VPN gateway intermittent drops for remote staff	New	P3	Jul 29, 16:14

The ServiceOps dashboard: stat tiles for Incidents, Requests, Changes and Open work across the top, followed by Assigned to me, SLA breached, SLA at risk and Recently updated panels.

The dashboard is the landing page after sign-in. The top row of stat tiles gives an at-a-glance count of open Incidents, Requests, Changes, and total Open work; each tile is a link straight into the matching filtered list, and the Incidents tile turns red with a P1/P2 breakdown whenever a high-priority incident is open. Below that, **Assigned to me** lists your own open tickets ordered by priority. **SLA breached** and **SLA at risk** only appear when there is something in them — an empty service desk means an empty dashboard, not two panels permanently reading "nothing here." **Recently updated** shows the last eight records touched anywhere you have visibility into, useful for picking back up after a context switch. Administrators can turn any of these panels on or off tenant-wide from **System settings**.

Incidents list

vo

ServiceOps

YOUR COMPANY

WORKSPACE

Dashboard

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

IT>

S

System Administrator

Admin

≡

Search ServiceOps

☆ ⌚ 🔔 🗑 ⚙

Incidents

8 records

New

↓ CSV

Print

Number or short description

Search

< Page 1 of 1 >

▼ All

NUMBER	OPENED	SHORT DESCRIPTION	CALLER	PRIORITY	STATE	CATEGORY	ASSIGNMENT GROUP	ASSIGNED TO	UPDATED
INC0000008	2026-07-29 16:14	APPSRV02 high CPU utilization	Seo-yeon Kim	P4	Closed	Infrastructure	Windows	Ana Silva	2026-07-29 16:14
INC0000007	2026-07-29 16:14	SAN01 disk predictive failure alert	Nisha Patel	P3	New	Infrastructure	SSD	Carl Nilsson	2026-07-29 16:14
INC0000006	2026-07-29 16:14	Exchange Online connector authentication errors	Bianca Reyes	P2	In Progress	Infrastructure	CoreApps	Saori Iwata	2026-07-29 16:14
INC0000005	2026-07-29 16:14	VPN gateway intermittent drops for remote staff	Seo-yeon Kim	P3	New	Infrastructure	Network	David Osei	2026-07-29 16:14
INC0000004	2026-07-29 16:14	Slow response times on Customer Portal during peak hours	Emma Wright	P2	In Progress	Software	Windows	Ana Silva	2026-07-29 16:14
INC0000003	2026-07-29 16:14	PORTALDB01 replication lag alert	Seo-yeon Kim	P3	Closed	Software	Database	Raj Singh	2026-07-29 16:14
INC0000002	2026-07-29 16:14	Payroll batch job did not complete overnight	Bianca Reyes	P1	Resolved	Software	Unix	Fatima Khan	2026-07-29 16:14
INC0000001	2026-07-29 16:14	Customer Portal login failures for a subset of users	Emma Wright	P2	Resolved	Software	Windows	Ana Silva	2026-07-29 16:14

The Incidents list workspace: a searchable, filterable, paginated table with Number, Opened, Short description, Caller, Priority, State, Category, Assignment group, Assigned to and Updated columns.

Every ticket list in ServiceOps (Incidents, Changes, Requests) shares the same list-workspace layout: a toolbar with a **New** button, a state filter, a free-text search box that matches both the record number and its short description, and pagination controls showing the current page and total record count. The table itself is sorted by most-recently-updated first and color-codes priority with a small dot (red for P1, amber for P2) so urgent work is visually distinct while scrolling a long list. Clicking any row's number opens that record's detail page.

Incident detail

vo

ServiceOps

YOUR COMPANY

WORKSPACE

Dashboard

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

System Administrator Admin

Incident

INC0000008

Updated Jul 29, 16:14

Reopen

INC0000008 is Closed and locked: only comments and notes can be added. Use Reopen to make other changes.

New

In Progress

Pending

Resolved

Closed

Number

INC0000008

Contact type

Self-service

* Caller

Seo-yeon Kim · s.kim@demo.serviceops.inv

State

Closed

Category

General

Impact

Medium

Subcategory

Urgency

Medium

Service offering

Not selected

Priority

P4

Configuration item

Search configuration items...

* Assignment group

Windows

Team manager

Kenji Nakamura

Notify

Email

Assigned to

Ana Silva

* Short description

APPSRV02 high CPU utilization

* Description

APPSRV02 high CPU utilization. Reported via Self-service.

Notes

Related records

Resolution information

Event history

Activity and work notes

Conversation and resolution evidence

No comments yet.

Add a work note or comment...

Post comment

Service commitments

Active SLA definitions and breach targets

No active service commitments.

An open incident record: a lifecycle stepper (New → In Progress → Pending → Resolved → Closed) across the top, a two-column form with State, Priority, Category, Assignment group and Assigned to fields, and an Event history timeline below.

The incident detail page is built on the same "record shell" used for every ITIL record type in ServiceOps: a header bar with the record number and permitted actions (Update, Resolve), a horizontal lifecycle stepper showing exactly which state the record is in and which states are still ahead, a dense two-column form for the operational fields, and a chronological Event history immediately below recording every field change with who made it and when. Only an active member of the incident's owning fulfillment team (or an administrator) can change these fields — other teams can open this same page to read it for cross-team awareness, but the form controls stay disabled for them.

Changes list

vo

ServiceOps

YOUR COMPANY

≡

Search ServiceOps

☆

🕒

🔔

🔗

⚙️

WORKSPACE

Dashboards

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

S

System Administrator Admin

➦

☰

Changes

4 records

New

↓ CSV

Print

Number or short description

Search

<

Page 1 of 1

>

▼ All

NUMBER	OPENED	SHORT DESCRIPTION	CALLER	PRIORITY	STATE	CATEGORY	ASSIGNMENT GROUP	ASSIGNED TO	UPD
CHG0000012	2026-07-29 21:15	emergency	System Administrator	P2	Awaiting Approval	General	Unix	Unassigned	2026-07-29 21:15
CHG0000011	2026-07-29 16:14	Deploy standard laptop imaging update	Ana Silva	P3	Closed	Infrastructure	Windows	Ana Silva	2026-07-29 16:14
CHG0000010	2026-07-29 16:14	Emergency firewall rule rollback on EDGE-FW01	David Osei	P2	Implemented	Infrastructure	Network	David Osei	2026-07-29 16:14
CHG0000009	2026-07-29 16:14	Upgrade PORTALDB01 to PostgreSQL 16.4	Raj Singh	P3	Scheduled	Infrastructure	Database	Raj Singh	2026-07-29 16:14

The Changes list workspace, filtered to a specific state, showing CHG-numbered records with their risk-bearing priority and assignment group columns.

Changes use the identical list layout as incidents, but every row here represents a controlled production modification rather than a service interruption. Look at the State column to distinguish a change that's still `Awaiting Approval` from one already `Approved` and scheduled, or already `Implemented`. Standard, Normal and Emergency changes are all shown together in this list — open the record itself to see which governance path applies (see "Changes" earlier in this manual for the approval rules each type follows).

Requests & catalog

ServiceOps
YOUR COMPANY

WORKSPACE

- Dashboard
- Visual task board
- My tasks
- Incidents
- Service requests
- Changes
- Service catalog**
- Knowledge
- All workspaces
- My approvals
- Notifications
- Org chart

LEADERSHIP

- Manager portal

OPERATIONS

- Problems

System Administrator Admin

EMPLOYEE CENTER

Service catalog

Request services, equipment, access, and support.

Access

Software Request

Request installation or license access for approved software.

Estimated delivery: 2 business days · Approval required · Fulfilled by Windows

Add request details (optional)

Request

Hardware

Laptop Request

Request a standard-issue laptop for a new or replacement device.

Estimated delivery: 5 business days · Approval required · Fulfilled by Windows

Add request details (optional)

Request

The service catalog: a grid of orderable catalog items (Laptop Request, Software Request) each with a description, estimated delivery time, approval requirement and fulfilling team, and a "Request" button.

ServiceOps
YOUR COMPANY

WORKSPACE

- Dashboard
- Visual task board
- My tasks
- Incidents
- Service requests
- Changes
- Service catalog
- Knowledge
- All workspaces
- My approvals
- Notifications
- Org chart

LEADERSHIP

- Manager portal

OPERATIONS

- Problems

System Administrator Admin

Requests
4 records

Open catalog **↓ CSV** **Print** REQ number **Search** **Page 1 of 1**

REQUEST	REQUESTED FOR	ITEMS	STATE	OPENED
REQ0000008	System Administrator	RITM0000008 Software Request	Open	Jul 29, 19:03
REQ0000003	System Administrator	RITM0000003 Laptop Request	Open	Jul 29, 19:01
REQ0000002	Olivier Dubois	RITM0000002 Software Request	Open	Jul 29, 16:14
REQ0000001	Emma Wright	RITM0000001 Laptop Request	Closed Complete	Jul 29, 16:14

The Requests & RITMs list: REQ-numbered containers with Requested for, Items, State and Opened columns.

The ****Service catalog**** page is where end users browse and order predefined items. Each card states up front whether approval is required and which team will fulfill it, so there are no surprises about turnaround time. Submitting the form creates a REQ (the order) and one RITM per item ordered; the ****Requests & RITMs**** list is where you track

those containers afterward, independent of the catalog itself.

Requested Item (RITM) detail

The screenshot displays the ServiceOps interface for a Requested Item (RITM). The left sidebar contains navigation links for Workspace, Dashboard, Visual task board, My tasks, Incidents, Service requests, Changes, Service catalog, Knowledge, All workspaces, My approvals, Notifications, Org chart, Leadership, Manager portal, Operations, and Problems. The main content area shows the RITM detail for RITM0000008, which is in the 'Approved' state. The lifecycle stepper indicates the progression from 'Awaiting Approval' to 'Open', 'Closed Complete', and finally 'Approved'. Key fields include the RITM Number (RITM0000008), Request ID (REQ0000008), Item type (Software Request), and the requester (System Administrator). Additional details show the item was opened on Jul 29, 2026, by the System Administrator, is in the 'Fulfillment' stage, and is due on Jul 31, 2026. Below this, the 'Fulfillment tasks (SCTASK)' panel lists tasks generated for fulfillment, with one task 'SCTASK0000004 - Fulfill Software Request' currently in 'Open' status.

A RITM detail page showing its lifecycle stepper (Awaiting Approval → Open → Closed Complete), Opened/Opened by fields, and a Fulfillment tasks (SCTASK) panel below.

Opening a REQ shows its constituent RITMs; opening a RITM shows this page. The lifecycle stepper here reflects the RITM's own three-stage journey — approval, open fulfillment, and completion — distinct from the parent REQ's own state. Below the header, the **Fulfillment tasks** panel lists every SCTASK generated to actually deliver the item, each independently assignable to a fulfillment team; the RITM itself only reaches `Closed Complete` once every one of its SCTASKs finishes.

Open work

vo

ServiceOps

YOUR COMPANY

WORKSPACE

Dashboard

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

S

System Administrator

Admin

Search ServiceOps

☆

🕒

🔔

🔄

⚙️

NEEDS ATTENTION

Open work

Every incident, change, and service request that isn't resolved or closed yet.

Export CSV

Print / Save as PDF

Open incidents & changes

NUMBER	SUMMARY	STATE	PRIORITY	UPDATED
CHG0000012	emergency	Awaiting Approval	P2	Jul 29, 21:15
CHG0000010	Emergency firewall rule rollback on EDGE-FW01	Implemented	P2	Jul 29, 16:14
INC0000006	Exchange Online connector authentication errors	In Progress	P2	Jul 29, 16:14
INC0000004	Slow response times on Customer Portal during peak hours	In Progress	P2	Jul 29, 16:14
CHG0000009	Upgrade PORTALDB01 to PostgreSQL 16.4	Scheduled	P3	Jul 29, 16:14
INC0000007	SAN01 disk predictive failure alert	New	P3	Jul 29, 16:14
INC0000005	VPN gateway intermittent drops for remote staff	New	P3	Jul 29, 16:14

Open service requests

REQUEST	REQUESTED FOR	STATE	OPENED
REQ0000008	System Administrator	Open	Jul 29, 19:03
REQ0000003	System Administrator	Open	Jul 29, 19:01
REQ0000002	Olivier Dubois	Open	Jul 29, 16:14

The Open work page: every incident, change and service request across the tenant that isn't resolved or closed yet, in one combined view.

This is the single cross-cutting view of everything still outstanding, combining incidents, changes and requests into one list rather than three separate ones. It is capped at 200 rows per section to stay responsive on a busy service desk — a truncation notice appears with links to the fully filterable Incidents/Changes lists if there's more to see. Use the priority filter (linked from the dashboard's P1/P2 alert) to narrow straight to the highest-urgency backlog.

My tasks

ServiceOps Complete Platform Manual

Page 17

vo ServiceOps
YOUR COMPANY

WORKSPACE

Dashboard

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

System Administrator
Admin

Search ServiceOps

WORK QUEUE

My tasks

Change, problem, event, and service catalog tasks assigned to you or your teams.

Assigned to me

NUMBER	TYPE	SUMMARY	STATE	TEAM	DUE
Nothing assigned to you right now.					

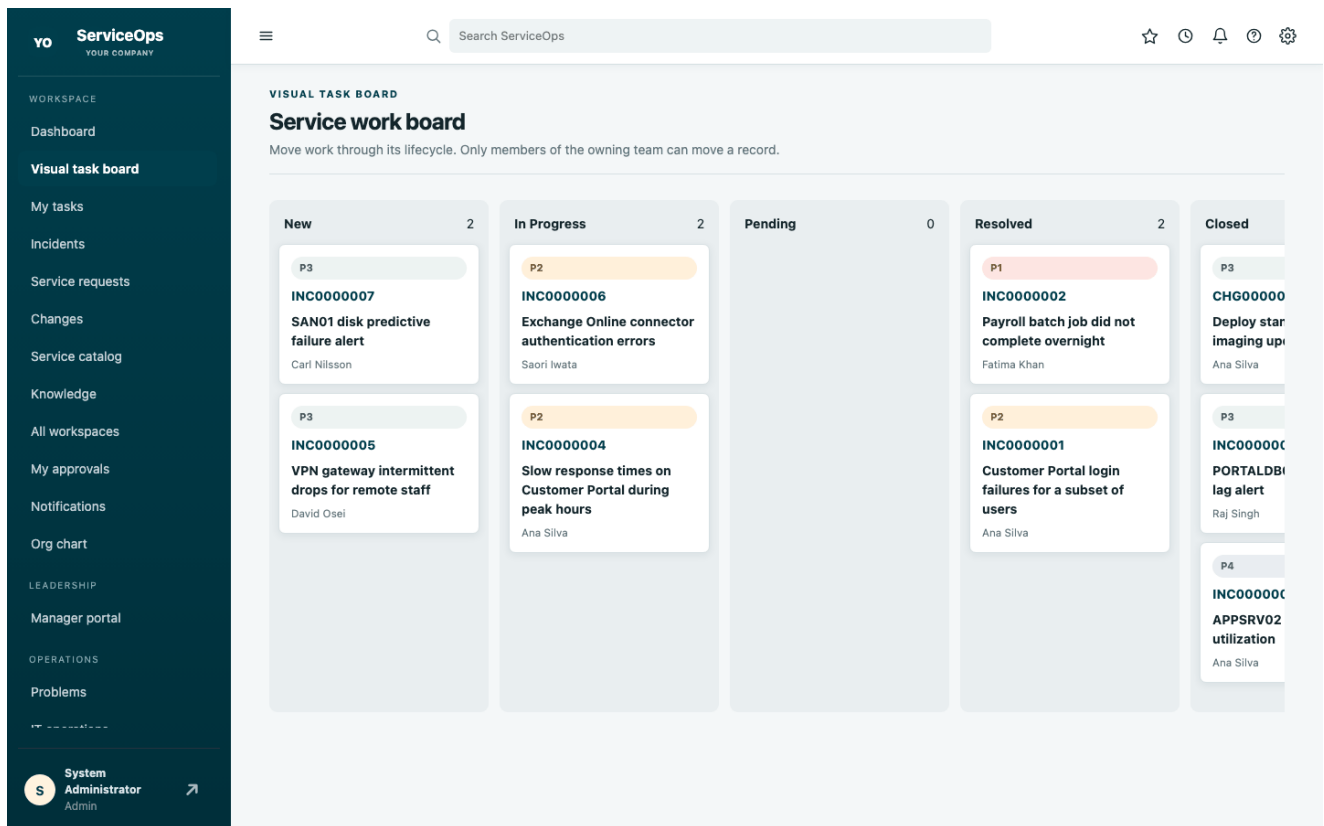
My teams' open tasks

NUMBER	TYPE	SUMMARY	STATE	TEAM	ASSIGNED TO	DUE
SCTASK0000004	SCTASK	Fulfill Software Request	Open	Windows	Unassigned	Jul 31, 19:03
CTASK0000012	CHANGE	Implementation	Pending	Unix	Unassigned	Aug 01, 06:15
SCTASK0000005	SCTASK	Fulfill Laptop Request	Open	Windows	Unassigned	Aug 03, 19:01
PTASK000101	PROBLEM	Reproduce failure in staging	Closed	Windows	Ana Silva	—
PTASK000102	PROBLEM	Review authentication service logs	Open	Windows	Ana Silva	—
PTASK000201	PROBLEM	Profile batch job runtime	Closed	Unix	Fatima Khan	—
PTASK000202	PROBLEM	Engage vendor support for Payroll Engine	Open	Unix	Fatima Khan	—
PTASK000301	PROBLEM	Review replication configuration	Closed	Database	Raj Singh	—
PTASK000302	PROBLEM	Validate network path to replica	Open	Database	Raj Singh	—
CTASK000901	CHANGE	Apply minor version upgrade	Open	Database	Raj Singh	—
CTASK000902	CHANGE	Run post-upgrade validation queries	Open	Database	Raj Singh	—
CTASK001001	CHANGE	Roll back rule set	Closed	Network	David Osei	—
CTASK001002	CHANGE	Confirm connectivity restored	Closed	Network	David Osei	—
CTASK001101	CHANGE	Push updated image to deployment share	Closed	Windows	Ana Silva	—
SCTASK0000001	SCTASK	Fulfill Laptop Request	Closed	Windows	Unassigned	—
SCTASK0000002	SCTASK	Fulfill Software Request	Open	Windows	Unassigned	—
SCTASK0000003	SCTASK	new	Open	CoreApps	Unassigned	—

The My tasks page: a queue of operational tasks and catalog fulfillment tasks assigned to the signed-in user, plus a secondary list of unassigned team work.

Agents and managers use this page as their personal work queue. **Assigned to me** is exactly that — CTASKs, PTASKs and SCTASKs with your user ID as the assignee, sorted so the nearest due date surfaces first. **Team tasks** below it shows everything else open against a team you belong to but not yet picked up by anyone, which is where you'd look to grab the next piece of unassigned work. The sidebar's "My tasks" link itself carries an amber badge whenever you have open work assigned, so you don't have to open the page to know something is waiting.

Visual task board



A Kanban-style visual task board with New, In Progress, Pending, Resolved and Closed lanes, each containing draggable ticket cards.

The task board gives a Kanban view of tickets you're permitted to manage. Dragging a card between lanes changes the underlying ticket's state exactly the same way editing the record's State field would — it is a different interaction model over the same governed transition rules, not a shortcut around them, so an invalid drag (e.g. skipping a required approval state) is rejected the same way an invalid form edit would be.

CMDB & service map

ServiceOps

Dashboard

Visual task board

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

Manager portal

Problems

System Administrator

IT OPERATIONS

CMDB & service map

Configuration items and the dependencies that connect business services.

Export CSV

Print / Save as PDF

19

32

19

Configuration Items

NAME	CLASS	ENVIRONMENT	STATUS	LIFECYCLE	CRITICALITY	LOCATION	OWNING TEAM	OWNER
Payroll Engine	Application	Production	Operational	In Use	High	DC1	Coreapps	System Administrator
Customer Portal	Business Application	Production	Operational	In Use	Critical	Business Service Catalog	Coreapps	System Administrator
Payroll Service	Business Application	Production	Operational	In Use	Critical	Business Service Catalog	Coreapps	System Administrator
Email & Collaboration	Business Service	Production	Operational	In Use	High	Business Service Catalog	Coreapps	System Administrator
PAYROLDB01	Database	Production	Operational	In Use	Critical	DC1	Database	System Administrator
PORTALDB01	Database	Production	Operational	In Use	High	DC1	Database	System Administrator
Exchange Online Connector	Middleware	Production	Operational	In Use	Medium	DC1	Coreapps	System Administrator
CORE-SW01	Network Device	Production	Operational	In Use	High	DC1 Rack 4	Network	System Administrator
EDGE-FW01	Network Device	Production	Operational	In Use	High	DC1 Rack 1	Network	System Administrator
LB01	Network Device	Production	Operational	In Use	High	DC1 Rack 2	Network	System Administrator
VPN-GW01	Network Device	Production	Operational	In Use	High	DC1 Rack 1	Network	System Administrator
APPSRV01	Server	Production	Operational	In Use	Medium	DC1	Windows	System Administrator
APPSRV02	Server	Production	Operational	In Use	Medium	DC1	Windows	System Administrator
APPSRV03	Server	Production	Operational	In Use	Medium	DC1	Windows	System Administrator
APPSRV04	Server	Production	Operational	In Use	Medium	DC1	Windows	System Administrator
DSERV01	Server	Production	Operational	In Use	Medium	DC1	Unix	System Administrator
BACKUP-NAS01	Storage	Production	Operational	In Use	High	DC1 Rack 7	SSD	System Administrator
SAN01	Storage	Production	Operational	In Use	High	DC1 Rack 6	SSD	System Administrator
Portal Web App	Web application	Production	Operational	In Use	High	DC1	Coreapps	System Administrator

Service relationships

Customer Portal — Depends on — Portal Web App

Portal Web App — Depends on — PORTALDB01

Portal Web App — Runs on — APPSRV01

PORTALDB01 — Runs on — APPSRV01

Portal Web App — Runs on — APPSRV02

PORTALDB01 — Runs on — APPSRV02

Payroll Service — Depends on — Payroll Engine

Payroll Engine — Depends on — PAYROLDB01

Payroll Engine — Runs on — APPSRV03

PAYROLDB01 — Runs on — APPSRV03

Payroll Engine — Runs on — DSERV01

PAYROLDB01 — Runs on — DSERV01

Email & Collaboration — Depends on — Exchange Online Connector

Exchange Online Connector — Runs on — APPSRV04

APPSRV01 — Connects to — CORE-SW01

APPSRV01 — Stores on — SAN01

APPSRV01 — Backed up by — BACKUP-NAS01

APPSRV02 — Connects to — CORE-SW01

APPSRV02 — Stores on — SAN01

APPSRV02 — Backed up by — BACKUP-NAS01

APPSRV03 — Connects to — CORE-SW01

APPSRV03 — Stores on — SAN01

APPSRV03 — Backed up by — BACKUP-NAS01

APPSRV04 — Connects to — CORE-SW01

APPSRV04 — Stores on — SAN01

APPSRV04 — Backed up by — BACKUP-NAS01

DSERV01 — Connects to — CORE-SW01

DSERV01 — Stores on — SAN01

DSERV01 — Backed up by — BACKUP-NAS01

CORE-SW01 — Connects to — EDGE-FW01

LB01 — Balances — APPSRV01

LB01 — Balances — APPSRV02

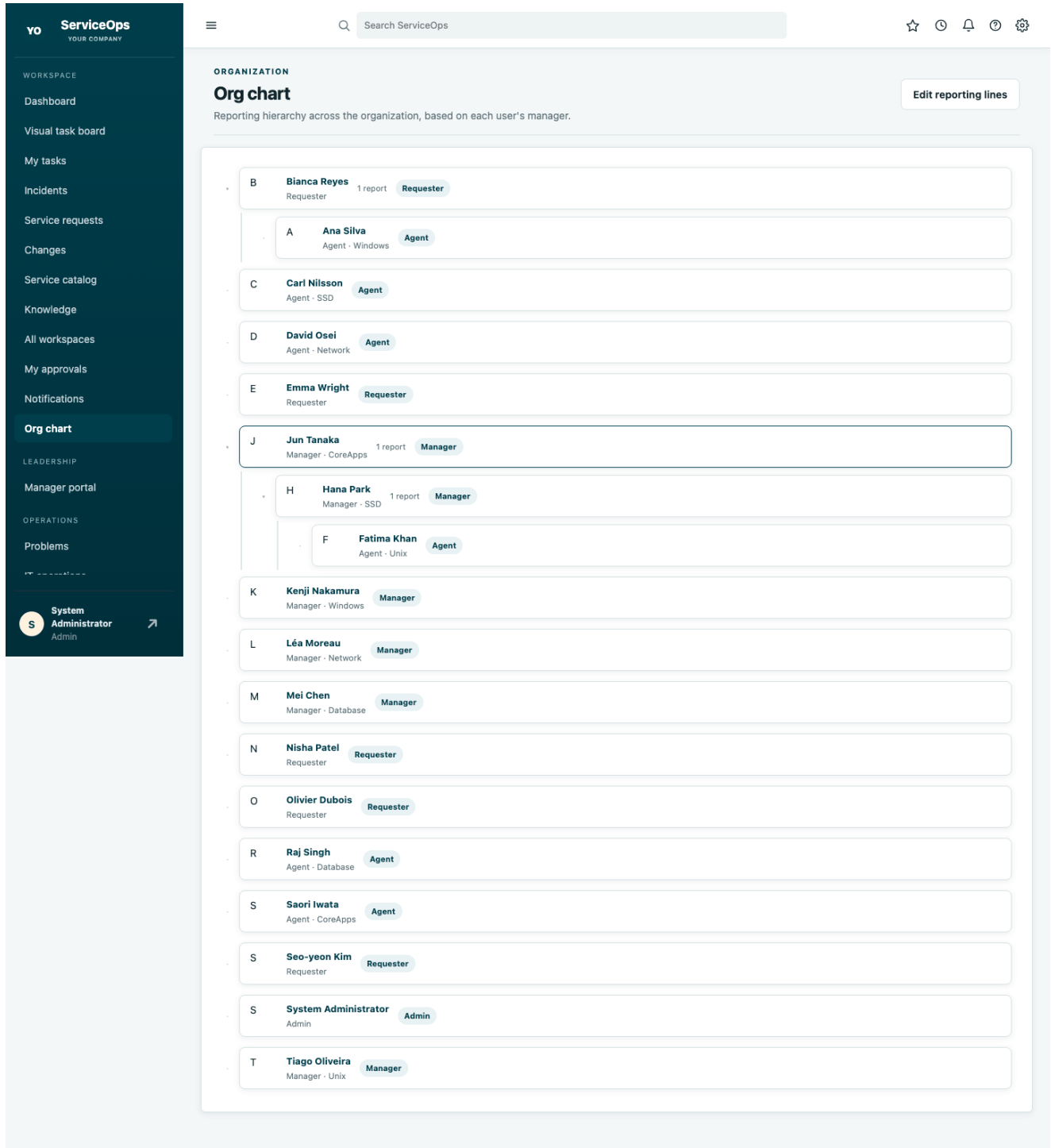
Add relationship

The CMDB page: a table of configuration items with Name, Class, Environment, Status, Lifecycle, Criticality, Location, Owning team and Owner columns, followed by a list of CI-to-CI relationships.

The Configuration Management Database is the system of record for every server, application and service ServiceOps tracks, and how they depend on one another. Each CI carries a lifecycle state (separate from its operational status —

a CI can be operationally `Operational` while its lifecycle state is `Planned` ahead of go-live, for example), a business-criticality rating used elsewhere in change-conflict detection, and an owning support team. Relationships beneath the CI table (e.g. "Customer Portal — Depends on → Payroll Database") are what powers change-conflict detection: scheduling a change against a CI that another in-flight change already touches, directly or through a relationship, surfaces a conflict warning before approval.

Org chart



A vertically indented org chart showing manager-to-report reporting lines, with each person's name, title, department and a role badge.

The org chart is generated automatically from each user's configured manager field — there's no separate org-chart data entry, so it's always consistent with **Users & roles**. Nodes with reports show a collapse toggle so a large organization can be explored level by level instead of one long scroll.

Manager portal

yo ServiceOps
YOUR COMPANY

WORKSPACE

Dashboard

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

S System Administrator Admin

Search ServiceOps

☆ ⌚ 🔔 ⌚ ⚙️

LEADERSHIP

Manager portal

Team workload, SLA exposure and performance across the IT fulfillment teams you manage.

Export CSV Print / Save as PDF

CoreApps

Manager: Jun Tanaka · 2 members

1 open incident · 0 open changes · 0 open tasks

MEMBER	STATUS	OPEN INCIDENTS	OPEN CHANGES	OPEN TASKS	RESOLVED (30D)	SLA BREACHED	SLA AT RISK
Jun Tanaka Manager	Active	0	0	0	0	0	0
Saori Iwata Agent	Active	1	0	0	0	0	0

Database

Manager: Mei Chen · 2 members

0 open incidents · 1 open change · 4 open tasks

MEMBER	STATUS	OPEN INCIDENTS	OPEN CHANGES	OPEN TASKS	RESOLVED (30D)	SLA BREACHED	SLA AT RISK
Mei Chen Manager	Active	0	0	0	0	0	0
Raj Singh Agent	Active	0	1	4	1	0	0

Network

Manager: Léa Moreau · 2 members

1 open incident · 1 open change · 2 open tasks

MEMBER	STATUS	OPEN INCIDENTS	OPEN CHANGES	OPEN TASKS	RESOLVED (30D)	SLA BREACHED	SLA AT RISK
David Osei Agent	Active	1	1	2	0	0	0
Léa Moreau Manager	Active	0	0	0	0	0	0

SSD

Manager: Hana Park · 2 members

1 open incident · 0 open changes · 0 open tasks

MEMBER	STATUS	OPEN INCIDENTS	OPEN CHANGES	OPEN TASKS	RESOLVED (30D)	SLA BREACHED	SLA AT RISK
Carl Nilsson Agent	Active	1	0	0	0	0	0
Hana Park Manager	Active	0	0	0	0	0	0

Unix

Manager: Tiago Oliveira · 2 members

0 open incidents · 0 open changes · 2 open tasks

MEMBER	STATUS	OPEN INCIDENTS	OPEN CHANGES	OPEN TASKS	RESOLVED (30D)	SLA BREACHED	SLA AT RISK
Fatima Khan Agent	Active	0	0	2	1	0	0
Tiago Oliveira Manager	Active	0	0	0	0	0	0

Windows

Manager: Kenji Nakamura · 2 members

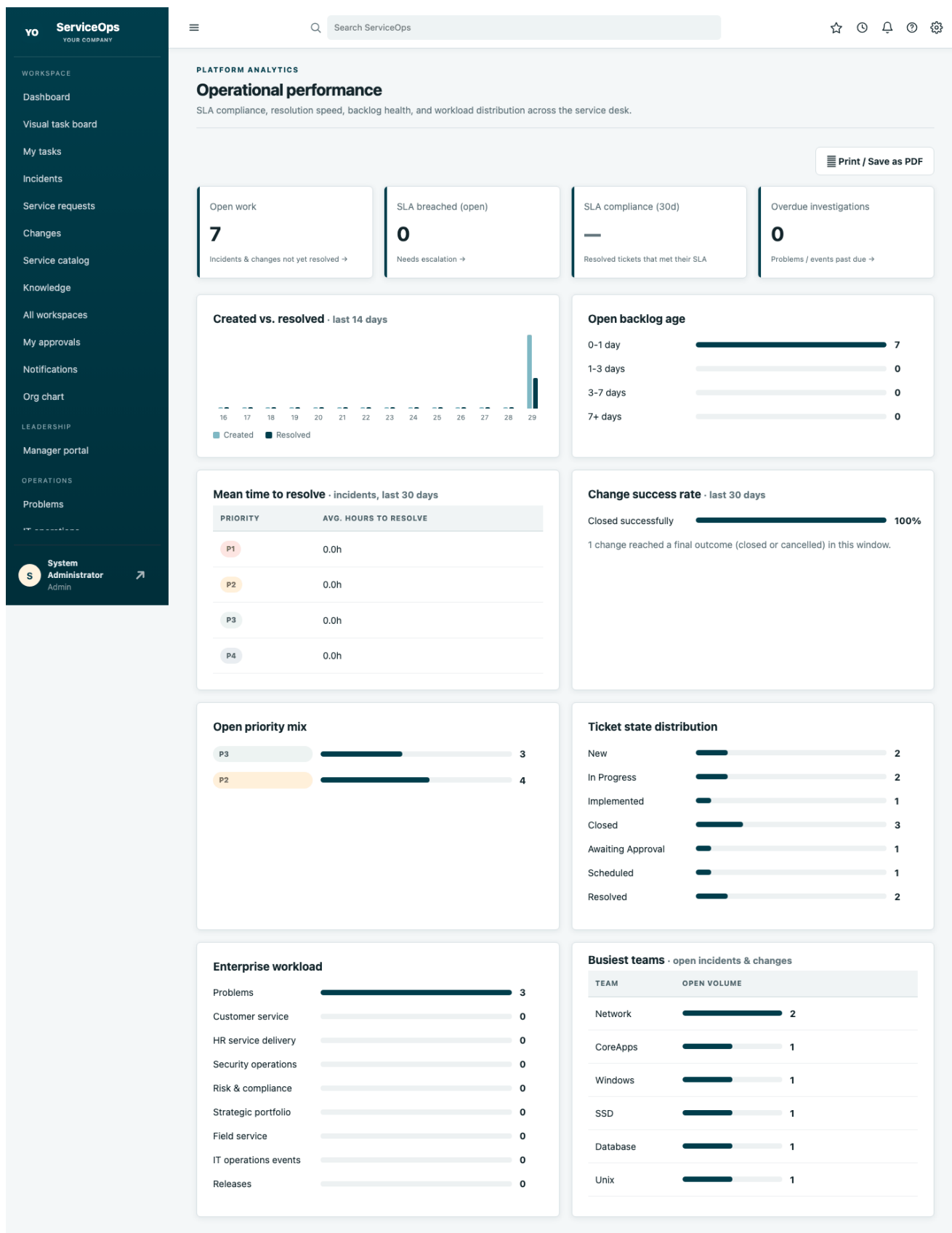
1 open incident · 0 open changes · 3 open tasks

MEMBER	STATUS	OPEN INCIDENTS	OPEN CHANGES	OPEN TASKS	RESOLVED (30D)	SLA BREACHED	SLA AT RISK
Ana Silva Agent	Active	1	0	3	3	0	0
Kenji Nakamura Manager	Active	0	0	0	0	0	0

The manager portal: one panel per team showing team-level open incident/change/task counts and SLA-breach totals, followed by a per-member table with status, workload and SLA columns.

This is the primary reporting surface for a team manager. Each team you manage gets its own panel; inside it, every team member has a row showing their active/inactive status, how many incidents/changes/tasks are currently open against them, how many tickets they've resolved in the last 30 days, and their personal SLA breached/at-risk counts. The ****Export CSV**** and ****Print / Save as PDF**** buttons at the top let this exact view be pulled into a spreadsheet or a printed report for a status meeting without re-deriving the numbers by hand.

Analytics



The analytics page: SLA compliance and breach/at-risk stat tiles, a 14-day created-vs-resolved trend chart, backlog aging buckets, mean-time-to-resolve by priority, change success rate, priority/state distribution bars, and a busiest-teams ranking.

Analytics is the tenant-wide operational-performance view, distinct from the manager portal's per-team/per-person focus. The created-vs-resolved trend chart is the fastest way to see whether the backlog is growing or shrinking week over week; backlog aging shows how long currently-open tickets have been sitting, which is often a better

early-warning signal than raw open counts; and mean-time-to-resolve by priority is the number most worth tracking over time as a service-desk health metric.

My approvals

ServiceOps
YOUR COMPANY

WORKSPACE

- Dashboard
- Visual task board
- My tasks
- Incidents
- Service requests
- Changes
- Service catalog
- Knowledge
- All workspaces
- My approvals**
- Notifications
- Org chart

LEADERSHIP

- Manager portal

OPERATIONS

- Problems

System Administrator Admin

GOVERNANCE

Approval chains

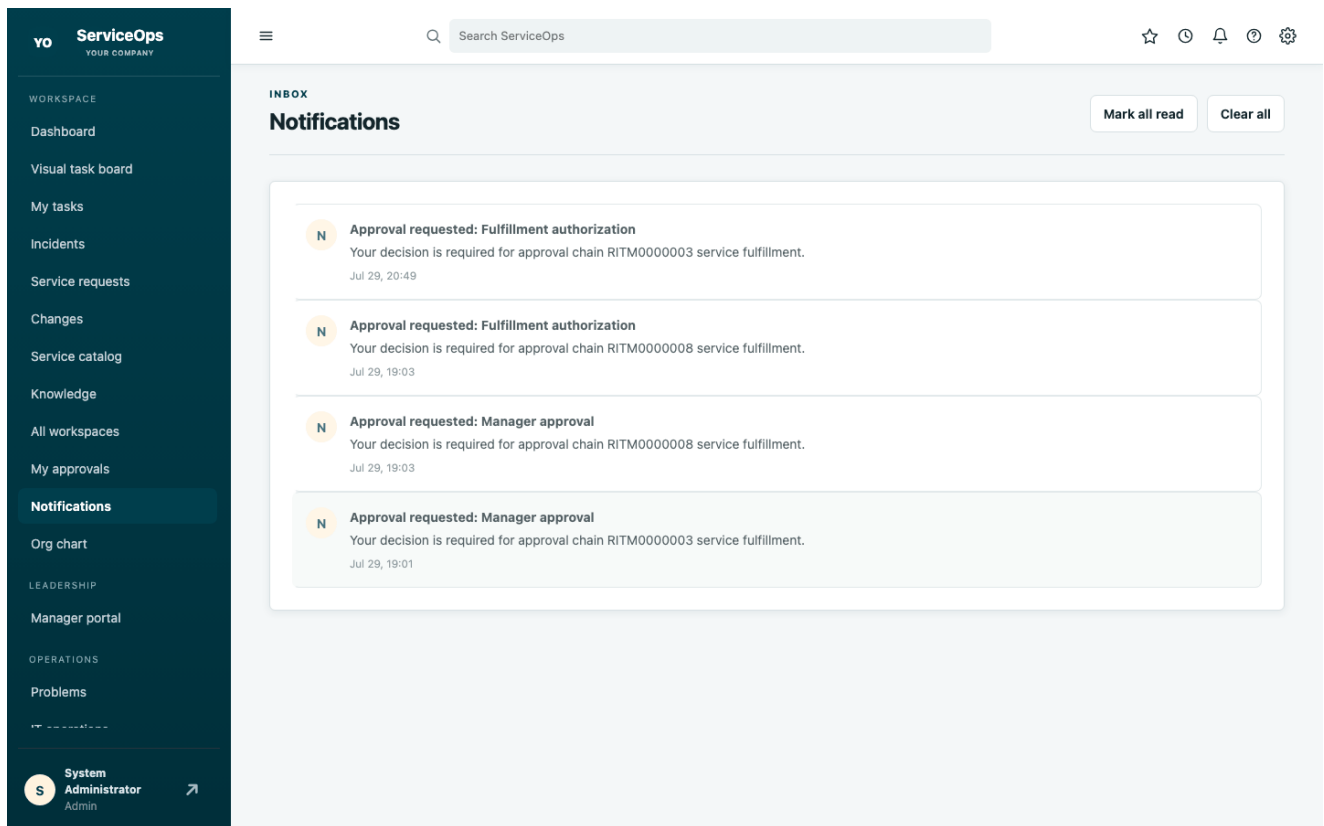
Sequential and parallel decisions with complete vote history.

CHAIN	TARGET	CURRENT STAGE	STATE	CREATED
CHG0000012 change authorization v1	TICKET #12	1 / 2	Running	Jul 29, 21:15
RITM0000008 service fulfillment	RITM #8	2 / 2	Approved	Jul 29, 19:03
RITM0000003 service fulfillment	RITM #7	2 / 2	Approved	Jul 29, 19:01

The My approvals page: pending approval chains grouped by target record, each showing its gates, the votes already cast, and inline approve/reject controls for the ones waiting on the signed-in user.

Anyone named as an approver — a team manager, a CCB member — sees their pending decisions here rather than having to remember to check every change or RITM individually. The sidebar's "My approvals" link carries the same amber badge treatment as "My tasks": if there's nothing waiting on you, there is no badge; if there is, the count is right there before you open the page.

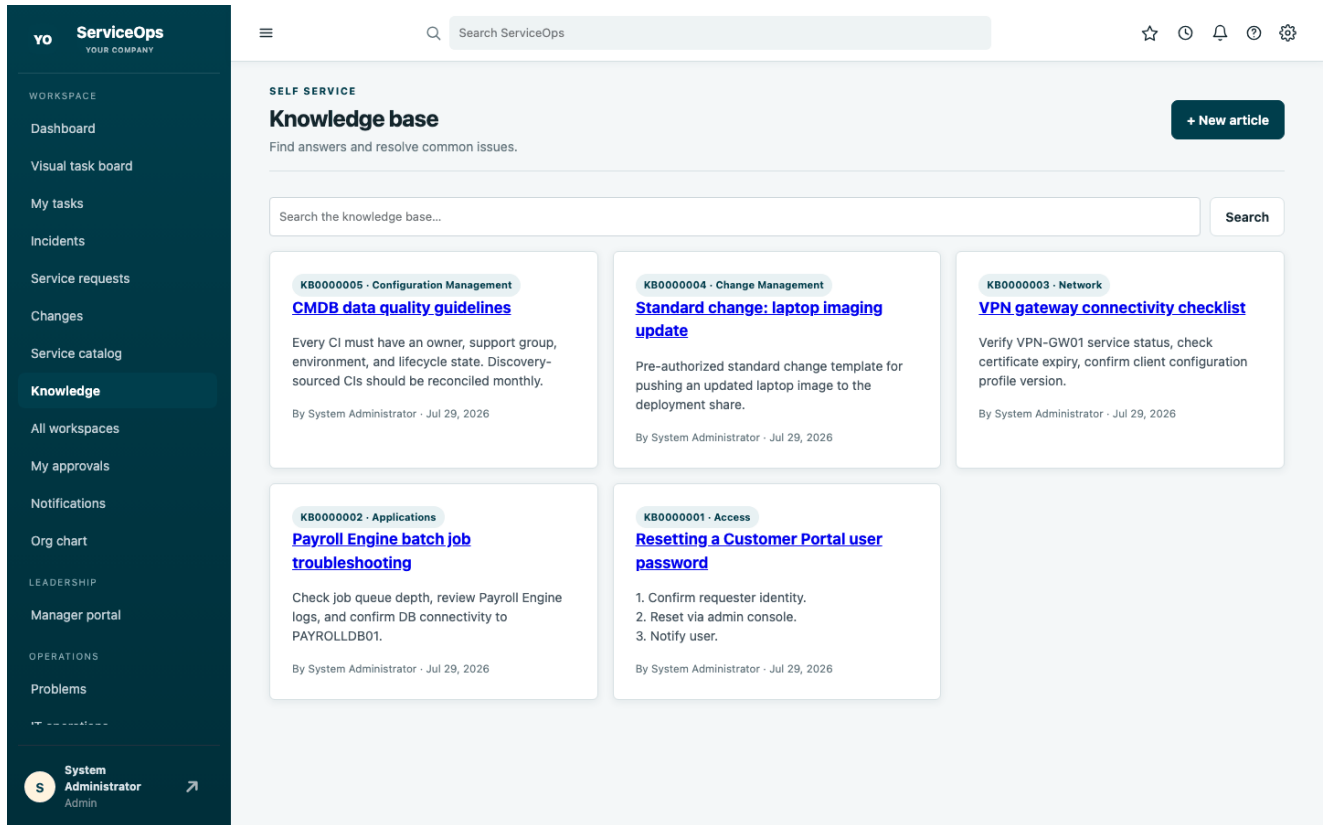
Notifications



The notifications inbox: a list of system notifications with unread items visually distinguished by a colored left border and accent dot.

Notifications are generated for the events you'd actually want to react to — an SLA breach on your ticket, a new approval request, a comment on something you're following. Unread items stay visually distinct until you click through to the record they refer to; opening the notifications page itself no longer silently marks everything read the moment you look at it.

Knowledge base



The knowledge base: a card grid of published articles with title, summary and category.

Search knowledge before opening a duplicate incident or RITM — a workaround or a known error entry here can resolve a user's problem immediately without creating new operational work at all. Problems (PRB records) link back to the knowledge article that documents their permanent fix once root cause is found, closing the loop between "we investigated this once" and "here's how to solve it fast next time."

Administration → System settings

ServiceOps

WORKSPACE

Dashboard

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

System administrator

PLATFORM ADMINISTRATION

System settings

Configure the deployed platform. Every change is validated and audited.

General

Appearance

Authentication

Security

Workflow

Dashboard

Email

Infrastructure

Company logo

General

Appearance

Authentication

Security

Workflow

Dashboard

Email

Infrastructure-owned configuration

Company logo

PNG only, maximum 5 MB. Recommended transparent canvas, up to 600 × 200 px.

Upload replacement PNG

Choose File

No file chosen

General

Settings marked "Live" apply without a restart. Others require a full application rollout.

Instance name

ServiceOps

Company name

Your Company

Support email

Appearance

Settings marked "Live" apply without a restart. Others require a full application rollout.

Primary brand color

Accent brand color

Audited change

Review authentication carefully before saving.

Validate and save settings

Comfortable

Authentication

Settings marked "Live" apply without a restart. Others require a full application rollout.

Enable local authentication

Enabled

Enable AD/LDAP

Enabled

LDAP server URI

LDAP bind password

Not configured

LDAP user filter

{0}(objectClass=user){1}(sAMAccountName={username}}

Use LDAP StartTLS

Enabled

Validate LDAP certificate

Enabled

LDAP group role mappings

0

Enable Keycloak

Restart required

Keycloak discovery URL

Restart required

Keycloak client ID

Restart required

Keycloak client secret

Not configured

Keycloak realm-role mappings

0

Security

Settings marked "Live" apply without a restart. Others require a full application rollout.

Enable HSTS

Enabled

Session lifetime in hours

8

Maximum upload size (MB)

20

Stream audit events to SIEM

Enabled

Failed logins before lockout

5

Lockout duration in minutes

15

REST API requests per minute (per client)

5

Workflow

Settings marked "Live" apply without a restart. Others require a full application rollout.

Default ticket priority

P3

Change freeze message

Synchronize parent incident state to children

Enabled

Dashboard

Settings marked "Live" apply without a restart. Others require a full application rollout.

Show "Assigned to me"

Enabled

Show SLA breached / at-risk widgets

Enabled

Show "Recently updated"

Enabled

SLA "at risk" warning window (hours)

4

Email

Settings marked "Live" apply without a restart. Others require a full application rollout.

Enable SMTP delivery

Enabled

SMTP host

SMTP port

587

Require SMTP STARTTLS

Enabled

SMTP username

SMTP password

Not configured

SMTP from address

Infrastructure-owned configuration

Displayed here for visibility. Change these through Compose or Helm so every instance remains consistent.

SETTING	EFFECTIVE VALUE	SOURCE OF CONTROL
Deployment profile	production	Cluster environment / Helm values
Database	db-5432@serviceops	ENVVAR@_URL / Kubernetes secret
Upload storage	Jpg/uploads	Cluster volume / Kubernetes PVC
Application replicas	Controlled externally	Cluster Compose / Helm
Ingress and TLS	Controlled externally	Reverse proxy / Kubernetes Ingress

The System settings page: a long form of tenant-wide configuration grouped into sections (branding, security, workflow, dashboard, email, identity providers), each field marked Live or Restart required.

Every tenant-wide behavior this manual describes as configurable — branding, security limits, dashboard panel visibility, SLA warning windows, the new REST API rate limit, LDAP/Keycloak connection details — is set from this one

page. Each field's badge tells you whether a change takes effect immediately (**Live**) or needs a full application restart/rollout to propagate to every running instance (**Restart required**) — this distinction matters most in a multi-replica Kubernetes deployment, where a Live setting is read fresh from the database by every request but a Restart-required identity-provider change must reach every pod consistently.

Users & roles

U Users User administration **New** **Search** **Administration home**

▼ All users 18 records

USER ID	NAME	EMAIL	ACTIVE	ROLE	DEPARTMENT	TEAM MEMBERSHIPS
a.silva	Ana Silva	a.silva@demo.serviceops.invalid	true	agent	Windows	Windows (member, Manual)
b.reyes	Bianca Reyes	b.reyes@demo.serviceops.invalid	true	requester	—	None
c.nilsson	Carl Nilsson	c.nilsson@demo.serviceops.invalid	true	agent	SSD	SSD (member, Manual)
d.osei	David Osei	d.osei@demo.serviceops.invalid	true	agent	Network	Network (member, Manual)
e.wright	Emma Wright	e.wright@demo.serviceops.invalid	true	requester	—	None
f.khan	Fatima Khan	f.khan@demo.serviceops.invalid	true	agent	Unix	Unix (member, Manual)
h.park	Hana Park	h.park@demo.serviceops.invalid	true	manager	SSD	SSD (manager, Manual)
j.tanaka	Jun Tanaka	j.tanaka@demo.serviceops.invalid	true	manager	CoreApps	CoreApps (manager, Manual)
k.nakamura	Kenji Nakamura	k.nakamura@demo.serviceops.invalid	true	manager	Windows	Windows (manager, Manual)
l.moreau	Léa Moreau	l.moreau@demo.serviceops.invalid	true	manager	Network	Network (manager, Manual)
m.chen	Mei Chen	m.chen@demo.serviceops.invalid	true	manager	Database	Database (manager, Manual)
n.patel	Nisha Patel	n.patel@demo.serviceops.invalid	true	requester	—	None
o.dubois	Olivier Dubois	o.dubois@demo.serviceops.invalid	true	requester	—	None
r.singh	Raj Singh	r.singh@demo.serviceops.invalid	true	agent	Database	Database (member, Manual)
s.iwata	Saori Iwata	s.iwata@demo.serviceops.invalid	true	agent	CoreApps	CoreApps (member, Manual)
s.kim	Seo-yeon Kim	s.kim@demo.serviceops.invalid	true	requester	—	None
admin	System Administrator	admin@example.local	true	admin	—	Service Desk (member, Manual)
t.oliveira	Tiago Oliveira	t.oliveira@demo.serviceops.invalid	true	manager	Unix	Unix (manager, Manual), Change Control Board (CCB approver, Manual)

The Users & roles administration page: a searchable table of every user with username, name, role, department, active state and team membership, and a link into each user's editable record.

Administrators manage the full user lifecycle here: role assignment, active/ inactive state, department, manager (which drives the org chart), and team membership. AD/LDAP-sourced accounts continue to reconcile their team membership automatically from directory group mappings on every login: an administrator editing team membership here only affects locally-managed accounts, or overrides that get reconciled again on the next directory sync.

Audit log

yoServiceOps
YOUR COMPANY

WORKSPACE

Dashboard

Visual task board

My tasks

Incidents

Service requests

Changes

Service catalog

Knowledge

All workspaces

My approvals

Notifications

Org chart

LEADERSHIP

Manager portal

OPERATIONS

Problems

System AdministratorAdmin

Search ServiceOps

SECURITY

Audit log

Append-only, request-correlated security and record activity.

Export verified evidence

Integrity keys

Historical keys remain encrypted for verification. Rotation never rewrites an audit event.

KEY ID	STATUS	ACTIVATED	RETIRED
Environment key is active. Rotate once to place versioned verification keys under governed storage.			
Confirmation		Type ROTATE	
Rotate audit key			

Retention and external evidence

Primary audit records are append-only and are never automatically purged. Dedicated active SIEM connections: 0.

Minimum retention days

2555

☐ Legal hold

☒ Require external immutable export

Save retention policy

Audit events

31 records

Action or target

Search

Page 1 of 1

Integrity: Not checked this view — verification hashes every event in the tenant's history and is only run on request. [Verify integrity now](#)

TIME	REQUEST	USER	ACTION	TARGET	DETAILS
2026-07-29 21:16	e8994f23	admin	login	admin	provider=local
2026-07-29 21:15	db5ee749	admin	create	CHG00000012	emergency
2026-07-29 21:15	db5ee749	admin	conflict check	CHG00000012	No conflict
2026-07-29 21:07	5fef42d2	admin	attach	CHG00000009	good.pdf
2026-07-29 21:06	a28ae9ce	admin	api client create	RateLimitTest	acting_user=admin; scopes=tickets.read
2026-07-29 21:06	4bb6cbfb	admin	login	admin	provider=local
2026-07-29 20:49	d84a84c4	admin	approved	RITM00000003 service fulfillment	Fulfillment authorization
2026-07-29 20:49	1696d684	admin	approved	RITM00000003 service fulfillment	Manager approval
2026-07-29 20:43	11bf47cd	admin	login	admin	provider=local
2026-07-29 19:28	2d866dff	admin	login	admin	provider=local
2026-07-29 19:20	e5f44b92	admin	login	admin	provider=local
2026-07-29 19:18	8f14ed19	admin	update	h.park	{\"before\": {\"name\": \"Hana Park\", \"email\": \"h.park@demo.serviceops.invalid\", \"role\": \"manager\", \"active\": true, \"department\": \"SSD\", \"manager\": \"None\"}, \"role\": \"manager\", \"active\": true}
2026-07-29 19:16	278483c4	admin	login	admin	provider=local
2026-07-29 19:06	4523bbfe	admin	update	f.khan	{\"before\": {\"name\": \"Fatima Khan\", \"email\": \"f.khan@demo.serviceops.invalid\", \"role\": \"agent\", \"active\": true, \"department\": \"Unix\", \"manager\": \"None\"}, \"role\": \"agent\", \"active\": true}
2026-07-29 19:06	2aca676b	admin	update	a.silva	{\"before\": {\"name\": \"Ana Silva\", \"email\": \"a.silva@demo.serviceops.invalid\", \"role\": \"agent\", \"active\": true, \"department\": \"Windows\", \"manager\": \"None\"}, \"role\": \"agent\", \"active\": true}
2026-07-29 19:03	57fa99ad	admin	approved	RITM00000008 service fulfillment	Fulfillment authorization
2026-07-29 19:03	757c8d7	admin	approved	RITM00000008 service fulfillment	Manager approval
2026-07-29 19:03	0e1d111e	admin	order	REQ00000008	RITM00000008: Software Request
2026-07-29 19:01	6a63dc2c	admin	order	REQ00000003	RITM00000003: Laptop Request
2026-07-29 18:59	0d86d814	admin	login	admin	provider=local
2026-07-29 18:59	5734f66c	system	login_failed	admin	attempts=1
2026-07-29 18:49	a7cc5557	admin	create	SCTASK00000003	RITM00000002: new
2026-07-29 17:00	4ee6e704	admin	configure	CCB approval authority	m.chen: revoked
2026-07-29 17:00	5176f9d1	admin	configure	CCB approval authority	l.moreau: revoked
2026-07-29 16:59	99a7f7f5	admin	configure	CCB approval authority	k.nakamura: revoked
2026-07-29 16:59	d186d3e6	admin	configure	CCB approval authority	j.tanaka: revoked
2026-07-29 16:59	1c41485f	admin	configure	CCB approval authority	h.park: revoked
2026-07-29 16:43	c82816cb	admin	login	admin	provider=local
2026-07-29 16:17	5d8573c3	system	login_failed	admin	attempts=1
2026-07-29 16:15	c9f84af6	admin	login	admin	provider=local
2026-07-29 16:15	46d018de	admin	login	admin	provider=local

The audit log: a chronological, tamper-evident table of every recorded action across the tenant — actor, action, target and timestamp.

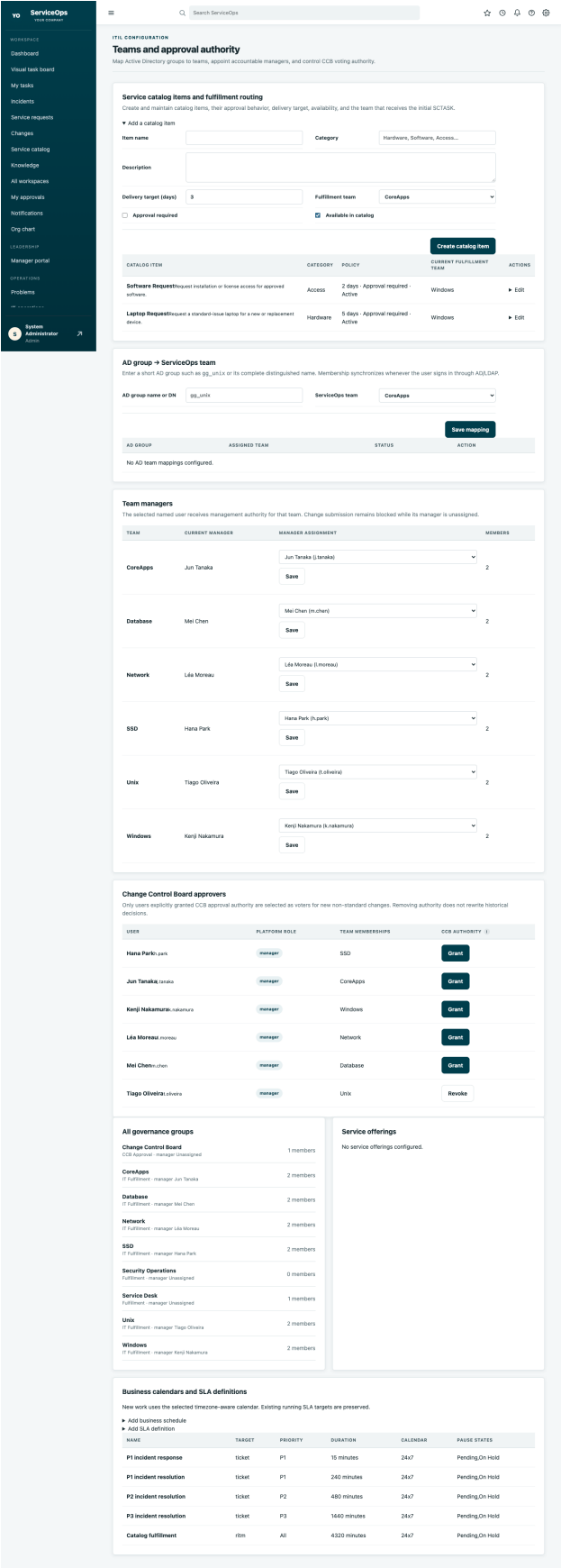
Every create, update, approval decision, and administrative action is recorded here with a cryptographic hash chain linking each event to the one before it, so the log itself can prove whether it has been tampered with. This is the

ServiceOps Complete Platform Manual

Page 30

evidence trail an external auditor or incident responder would ask for first; see "Ticket history and change reapproval" earlier in this manual for how per-record history relates to this tenant-wide audit stream.

ITIL configuration



The ITIL configuration page: administrator controls for catalog item fulfillment routing, SLA definitions, business calendars, and CCB/team-manager assignment.

This is where the process framework itself is configured, as opposed to day-to-day operational work: which team fulfills which catalog item by default, what an SLA's target duration and business calendar are, and who holds CCB approval

authority. Changes made here govern all future tickets and requests; they do not retroactively alter SLA targets already attached to an in-flight ticket.

3B. Common task walkthroughs

Report and resolve an incident

1. Click **Report an incident** on the dashboard, or **New** from the Incidents list.
2. Provide a concise title, observable symptoms, category/subcategory and business impact/urgency; ServiceOps calculates priority from impact and urgency automatically.
3. Submit — the incident is created in state `New` and routed to its owning fulfillment team.
4. An agent from that team opens the record (see the Incident detail screenshot above), sets it `In Progress`, and works the issue: comments, attachments, checklist items and CI links are all recorded in the Event history as they happen.
5. Once service is restored, the agent sets state to `Resolved`; a manager or the same agent later moves it to `Closed` once confirmed with the caller.

Submit a Normal change through approval

1. From the Changes list, click **New**, select change type `Normal`, and complete the required implementation plan, test plan, backout plan, planned window, affected CI and owning team.
2. On submission ServiceOps runs conflict detection against other in-flight changes touching the same CI or a related one, and creates a two-stage approval chain: the owning team's manager first, then the CCB.
3. The change sits in `Awaiting Approval` until both gates are satisfied — visible on **My approvals** (see the amber sidebar badge) for anyone named as an approver.
4. Once approved, the change moves to `Approved` and its change tasks (CTASK) can begin; a required CTASK still open blocks the change from completing.
5. Editing a material field (plan, risk, affected CI, schedule) after approval automatically supersedes the existing chain and starts a new one — an old approval is never silently applied to a changed plan.

Order an item from the service catalog

1. Open the **Service catalog** page, pick an item, add any requested details, and click **Request**.
2. ServiceOps creates a REQ container and one RITM for the item; if the item requires approval, an approval chain starts (manager, then the fulfilling team) before any fulfillment work begins.
3. Track status from **Requests & RITMs** — open the RITM itself to see its own lifecycle stepper and the SCTASKs doing the actual fulfillment work.
4. The RITM reaches `Closed Complete` once every SCTASK finishes; the parent

REQ reaches its own terminal state once every RITM in it does.

Review team performance as a manager

1. Open **Manager portal** from the Leadership section of the sidebar.
2. Each team you manage has its own panel with team-level open-work and SLA-breach totals, and a per-member table below it.
3. Use the per-member columns (open incidents/changes/tasks, 30-day resolved count, SLA breached/at-risk) to spot who's overloaded or who has SLA risk building up before it becomes a breach.
4. Use **Export CSV** to pull the exact table into a spreadsheet, or **Print / Save as PDF** to produce a printable report for a status meeting — both reflect exactly what's on screen.

4. Deployment decision

Environment	Application	Database	Upload storage
Single server	Docker Compose	Bundled PostgreSQL	Docker volume
Single production server	Docker Compose behind HTTPS proxy	Bundled or external PostgreSQL	Docker volume with backups
Enterprise production	Kubernetes/Helm, 3+ replicas	Managed HA PostgreSQL	RWX CSI volume or object-storage extension

Production Kubernetes must use an externally operated, highly available PostgreSQL service.

5. Docker installation

Run `./serviceops install web`, open `http://127.0.0.1:8090`, configure the profile, database, identity providers, and listener, validate all checks, confirm, and deploy. Keep the terminal open until the browser reports a healthy deployment.

Use `./serviceops status`, `health`, `logs`, `backup`, `restore`, and `doctor` for lifecycle operations. Put Caddy, NGINX, or an enterprise load balancer in front of the loopback-bound application and terminate TLS there.

6. Kubernetes prerequisites

- Kubernetes 1.27 or newer with at least three worker nodes for HA.
- Helm 3, kubectl, a default-deny-capable CNI, and a CSI storage provider.
- A private image registry and immutable ServiceOps image tag or digest.
- External HA PostgreSQL with TLS, automated backups, and point-in-time recovery.
- RWX upload storage when application replicas exceed one.
- An ingress controller, DNS, and automated TLS certificate management.
- Metrics Server for HPA; cluster monitoring and log aggregation.
- A secrets controller or external vault for steady-state secret rotation.

7. Kubernetes installation

1. Build, scan, sign, and push an immutable image.
2. Copy ``deploy/kubernetes/values-production.example.yaml`` to ``deploy/kubernetes/values-production.yaml``.
3. Set registry, immutable tag, ingress, storage class, replicas, identity endpoints, and role mappings.
4. Run ``./serviceops install kubernetes --preflight``.
5. Run ``./serviceops install kubernetes``.
6. Confirm rollout, Helm test, ingress TLS, login, record creation, upload, approval, notification, and audit behavior.

The installer labels the namespace for Restricted Pod Security, creates secrets from a protected temporary file, uses ``helm upgrade --install --atomic --wait``, waits for rollout, and runs the packaged ``/ready`` test.

8. Kubernetes chart controls

- Non-root UID/GID, RuntimeDefault seccomp, all capabilities dropped.
- Read-only root filesystem and disabled service-account-token mounts.
- Startup, readiness, and liveness probes with distinct semantics.
- Rolling update with zero unavailable replicas.
- Topology spreading across zones and hosts.
- PodDisruptionBudget and optional HPA.
- Resource requests and limits.
- NetworkPolicy for ingress and required egress ports.
- Persistent upload claim and optional ingress/TLS.
- JSON schema validation and a Helm test hook.

Tune topology keys to the labels present in the target cluster. A PDB protects only against voluntary disruptions; it does not protect against node, zone, or application failure.

9. Identity configuration

Local administrator

Keep local authentication enabled for one vaulted break-glass administrator. Test it quarterly under an approved access procedure. Rotate after every use.

AD/LDAP

Use LDAPS or StartTLS, certificate validation, a least-privilege read-only bind identity, a narrow base DN, an escaped username filter, and explicit group-role mappings. Validate bind, search, user bind, disabled-user behavior, group mapping, certificate expiry, and directory outage behavior.

Keycloak

Use a confidential OIDC client, authorization-code flow, exact HTTPS redirect URI, short-lived tokens, approved realm-role claims, client-secret rotation, and restrictive redirect/web-origin settings. Validate login, logout, expired session, revoked user, missing email, role changes, and provider outage.

10. Post-deployment system settings

Administrators can open **Administration → System settings** after deployment to change the platform name, company name, PNG logo, primary and accent colors, support identity, display defaults, LDAP, Keycloak, encrypted provider secrets, security limits, workflow defaults, and notification identity.

Each field is marked either **Live** or **Restart required**. Live settings are read from PostgreSQL by every request. Restart-required identity-client changes must be followed by a complete Compose restart or Kubernetes rollout so every application instance receives the same client configuration.

Database topology, replicas, storage, ingress, and TLS remain controlled by Compose or Helm and are displayed read-only. This prevents one pod from silently diverging from the declared infrastructure.

Sensitive values are encrypted before storage and are never returned to the browser. Set a durable `SETTINGS_ENCRYPTION_KEY` during installation; changing or losing that key makes existing encrypted settings unreadable.

Priority and SLA policy

Ticket priority is calculated from impact and urgency using the validated, Git-backed `config/priority_matrix.json`. Agents cannot silently override the result. A manager or administrator may select a different priority only while supplying an auditable reason of at least ten characters.

Administrators manage business calendars and SLA definitions under **ITIL configuration**. Calendars use IANA timezone names, explicit business weekdays and opening hours, plus named excluded holiday dates. An SLA without a calendar remains a 24x7 wall-clock commitment. A calendar change applies to future SLA attachments; it does not silently rewrite targets already in flight.

The worker detects newly breached commitments, writes one breach event to the SLA evidence stream, records the breach in the ticket history, and notifies the owning-team manager and assigned engineer through the durable outbox. Monitor the worker continuously; a stopped worker delays escalation but does not lose the stored target or breach state.

Declarative workflow operations

Workflow source is controlled in `config/workflows.json`. On startup ServiceOps validates the package and publishes a new immutable PostgreSQL runtime version only when the canonical specification changes. Administrators can inspect the deployed hash and versions, redeploy the packaged source, and run a mutation-free simulation under **Administration → Workflows**.

The supported foundation accepts `ticket.state_entry` events, equality, inequality, membership and empty-value conditions over explicitly allowed ticket fields, plus three actions: add ticket history, notify the requester, and notify the owning-team manager. Arbitrary administrator scripts and unsupported fields/actions fail package validation.

State transitions enqueue a unique correlated job in the same transaction as the operational change. The worker claims jobs using PostgreSQL coordination, records the exact published version and masked structured input/output, and commits actions atomically. Failures use bounded exponential retry and enter `Dead` after five attempts. Monitor both workflow jobs and execution evidence from the administration page; investigate dead jobs before replaying them.

Release 1.13.0 extends the runtime with durable waits, per-action execution evidence, manual/API/SLA-breach triggers, per-workflow rate limits, controlled dead-job replay, and safe terminal compensation. A wait commits its cursor and resume timestamp to PostgreSQL; after restart the worker continues at the next action and does not repeat completed steps.

API workflow triggers require the explicit `workflows:execute` scope, the acting user's operational permission for the ticket, and an idempotency key. Rate-limited jobs are delayed without consuming a failure attempt. Failed jobs retry five times; compensation runs only after the terminal failure and is currently restricted to an auditable ticket-history action because delivered notifications cannot truthfully be undone.

Release 1.13.0 does not yet claim scheduled recurrence, reusable subflows, general rollback, broad record mutation, concurrency quotas, or complete multi-environment configuration promotion. Those remain governed backlog work.

Release 1.14.0 adds reusable subflows to workflow package schema v2. Subflow references are validated before deployment, unknown dependencies fail closed, and dependency cycles are rejected. Valid subflows are expanded into the immutable published workflow specification, so runtime execution never depends on mutable external fragments.

Administrators configure recurring ticket schedules under **Administration → Workflows**. Each schedule is tenant-scoped, targets one ticket, uses a bounded minute interval, and can be enabled or disabled without deleting its evidence. Concurrent workers claim due schedules with PostgreSQL row locking. The event and next-run advancement commit together. If the system was offline for several intervals, it emits one event and advances directly to the next future interval instead of creating a catch-up storm.

Calendar expressions, blackout windows, package dependencies, per-tenant concurrency quotas and production-scale scheduler failure testing remain governed backlog work.

Bootstrap credential retirement

ServiceOps reads `ADMIN_PASSWORD_FILE` before the legacy `ADMIN_PASSWORD` environment value. Kubernetes separates the bootstrap credential from runtime secrets and never injects it into workers. Compose workers likewise receive no bootstrap administrator credential.

After the first successful installation:

1. Sign in as the local administrator.
2. Use **Administration → Change password** to rotate the initial credential

into a unique vault-managed password. This increments the account authentication version and invalidates every other browser session.

3. Store the new credential in the organizational emergency-access vault.
4. Run `./serviceops retire-bootstrap-secret`, verify the active administrator,

type the explicit confirmation, and wait for the recreated containers to pass health checks.

5. Run `./serviceops doctor`; the worker-secret isolation check must pass.

The retirement operation clears only bootstrap injection from `.env`; it does not erase or reset the administrator's salted password hash in PostgreSQL.

Release evidence

Tagged releases run `.github/workflows/supply-chain.yml`. Every referenced GitHub Action is pinned to a full commit SHA. The gate runs the complete test suite, builds with maximum provenance, blocks fixable HIGH and CRITICAL operating-system or library vulnerabilities, generates a CycloneDX image SBOM, and publishes only after successful validation. The resulting digest is signed keylessly with GitHub OIDC and receives GitHub SLSA build-provenance and SBOM attestations. Registry digest inspection and `gh attestation verify` must both succeed.

Kubernetes values must provide `image.digest`; application, worker and migration workloads use `repository@sha256:digest`, never a mutable tag. The installer requires `SERVICEOPS_GITHUB_ORGANIZATION`, installs the pinned Sigstore Policy Controller and GitHub trust policy, and labels the ServiceOps namespace for enforcement. Unsigned or untrusted matching organization images are rejected at admission. Run `python tools/verify_supply_chain.py` locally to detect release-policy drift.

Runtime Python and bundled PostgreSQL base images are pinned by digest and Python dependencies use exact versions. Generate the release record with:

```
python tools/release_evidence.py \
--version 1.26.3 \
--image serviceops-app:1.26.3 \
--output release-evidence/serviceops-1.26.3.json
```

The output records Git state, SHA-256 hashes for non-ignored source files, a combined manifest hash, image reference and CycloneDX component inventory. A dirty-tree marker is evidence, not approval. External vulnerability scanning, image signing, provenance publication, registry verification and admission enforcement remain required before organizational production approval.

11. Security operations

- Enforce TLS externally and enable HSTS only after HTTPS is proven.
- Store secrets in a vault; never commit `.env``, values secrets, or kubeconfig.
- Restrict namespace RBAC and database privileges.
- Scan source, dependencies, image, manifests, and exposed endpoints in CI.
- Sign images and enforce admission verification where supported.
- Forward application, ingress, Kubernetes audit, database, and identity logs.
- Alert on repeated login failures, admin changes, approval anomalies, SLA breaches, crash loops, readiness loss, saturation, and storage pressure.
- Patch base images and dependencies under change control.

12. Backup and recovery

Back up PostgreSQL and uploads as one recovery set. Encrypt backups, store them outside the cluster, apply retention and immutability, and record restore test evidence. For managed PostgreSQL enable PITR. Snapshotting a running database volume alone is not a proven logical backup.

Quarterly recovery test:

1. Run `./serviceops rehearse-recovery`` and `./serviceops rehearse-pitr``; retain both evidence records.
2. Restore into an isolated environment.
3. Restore database to the selected recovery point.
4. Restore uploads from the matching recovery set.
5. Deploy the matching immutable application version.
6. Validate counts, attachments, identities, approvals, audit, and critical workflows.
7. Record actual RPO/RTO and corrective actions.

The logical recovery command deliberately reports ``pitr_proven=false``: ``pg_dump`` cannot be replayed with WAL. The separate PITR rehearsal uses ``pg_basebackup``, continuous WAL archiving, and a named recovery target on disposable clusters, and reports ``pitr_proven=true`` only after boundary and ServiceOps integrity checks pass.

13. Upgrades and rollback

Run `./serviceops rehearse-upgrade`` first. It creates a verified rollback set and validates the candidate image and migration against an isolated clone while the source remains healthy. Back up first. Review release notes and schema compatibility. Render and lint the Helm chart, deploy to staging, run workflow regression and load tests, then use ``helm upgrade --install --atomic --wait``. Observe error rate, latency, readiness, database health, and queues. ``--atomic`` rolls Kubernetes resources back when the upgrade fails, but database schema/data rollback still requires a release-specific tested procedure.

14. Monitoring and SLOs

Monitor availability, request rate, latency percentiles, HTTP errors, worker saturation, pod restarts, readiness, CPU/memory throttling, database connection usage, query latency, replication/backup health, volume capacity, login errors, notification failures, and SLA breach rate. Define business-approved SLOs and page only on actionable symptoms.

15. Incident response

Declare severity and incident commander, preserve evidence, stabilize service, communicate on a fixed cadence, use tested rollback/failover, validate recovery, and create a blameless problem record with corrective actions. Never delete audit or operational evidence during response.

16. Production acceptance checklist

- Immutable, scanned image from trusted registry
- External HA PostgreSQL with TLS, PITR, and restore test
- Three or more application replicas across failure domains
- RWX persistent uploads and tested restore
- TLS ingress, DNS, HSTS, security headers
- Restricted Pod Security and least-privilege RBAC
- Network policies validated with the actual CNI and endpoint topology
- LDAP/Keycloak end-to-end tests and vaulted break-glass account
- Monitoring, logs, alert routing, dashboards, and runbooks
- Load, soak, failover, node-drain, rollback, and disaster-recovery tests
- Security review, penetration test, and formal go-live approval